

Rate-monotonic scheduling

- Rate-monotonic scheduling (RMS), introduced by Liu and Layland [Liu73], was one of the first scheduling policies developed for real-time systems and is still very widely used.
- RMS is a static scheduling policy.
- It turns out that these fixed priorities are sufficient to efficiently schedule the processes in many situations.
- The theory underlying RMS is known as rate monotonic analysis (RMA).

This theory,as summarized below, uses a relatively simple model of the system.

- All processes run periodically on a single CPU.
- Context switching time is ignored
- There are no data dependencies between processes.
- The execution time for a process is constant.
- All deadlines are at the ends of their periods.
- The highest-priority ready process is always selected for execution.

Liu and Layland[Liu73] proved that the RMA priority assignment is optimal using critical-instant analysis.

We define the

Response time of a process as the time at which the process finishes.

The critical instant for a process is defined as the instant during execution at which the task has the largest response time.

- **Rate monotonic scheduling** is a priority algorithm that belongs to the static priority scheduling category of [Real Time Operating Systems](#).
- It is preemptive in nature.
- The priority is decided according to the cycle time of the processes that are involved.
- If the process has a small job duration, then it has the highest priority.
- Thus if a process with highest priority starts execution, it will preempt the other running processes.
- The priority of a process is inversely proportional to the period it will run for.

- A set of processes can be scheduled only if they satisfy the following equation:

$$U = \sum_{i=1}^n \frac{C_i}{T_i} \leq n(2^{1/n} - 1)$$

Where n is the number of processes in the process set, C_i is the computation time of the process, T_i is the Time period for the process to run and U is the processor utilization.

Rate Monotonic Scheduling (RMS)

- **Schedulability check (off-line)**
 - A set of n tasks is schedulable on a uniprocessor by the RMS algorithm if the processor utilization (utilization test)

$$\sum_{i=1}^n c_i/p_i \leq n(2^{1/n} - 1).$$

c_i is the execution time and p_i is the period,
The term $n(2^{1/n} - 1)$ approaches $\ln 2$, (≈ 0.69 as $n \rightarrow \infty$).

- This condition is sufficient, but not necessary.

Example 1

Processes	Execution Time (C)	Time period (T)
P1	3	20
P2	2	5
P3	2	10

$$n(2^{1/n} - 1) = 3(2^{1/3} - 1) = 0.7977$$

$$U = 3/20 + 2/5 + 2/10 = 0.75$$

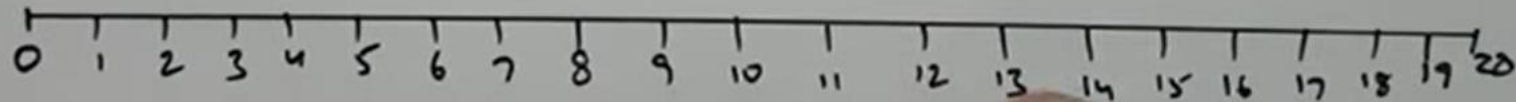
It is less than 1 or 100% utilization. The combined utilization of three processes is less than the threshold of these processes which means the above set of processes is schedulable and thus satisfies the above equation of the algorithm.

Rate Monotonic Scheduling

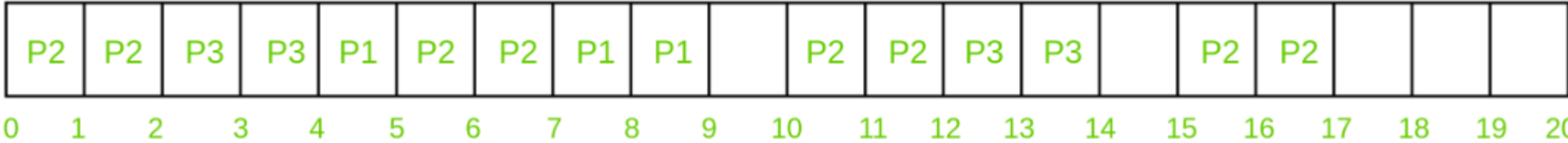
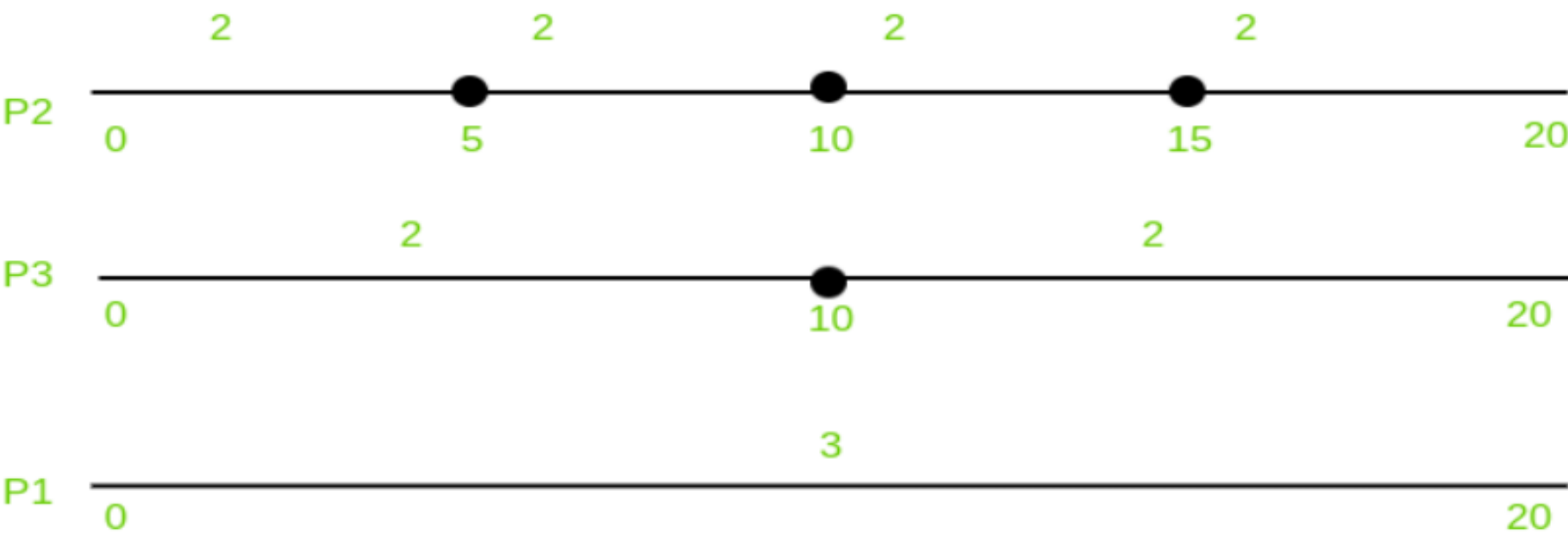
	<u>capacity</u>	<u>Period</u>
T_1	3 (C)	20 (P)
T_2	2	5
T_3	2	10

① $L.C.M(20, 5, 10) = 20$

M.V.R.



Representation and flow is as follows:



- The above figure says that Process P2 will execute two times for every 5 time units, Process P3 will execute two times for every 10 time units and Process P1 will execute three times in 20 time units. This has to be kept in mind for understanding the entire execution of the algorithm below.



Example 2

g - Rate Monotonic Algorithm Example

Process	Burst Time	Arrival Time	Period
P1	2	0	10
P2	1	0	5
P3	5	0	30
P4	2	0	15

$$N(2^{\frac{1}{N}} - 1)$$

$$U_i = \frac{t_i}{p_i}$$

Check whether it is possible to schedule the processes,
so that all of them meet their deadlines

$$N = 4$$

(4 processes)

$$4(2^{\frac{1}{4}} - 1) = 0.7568$$

Process	Burst Time	Arrival Time	Period
P1	2	0	10
P2	1	0	5
P3	5	0	30
P4	2	0	15

— $U_1 = \frac{2}{10} = \mathbf{0.2}$

$U_2 = \frac{1}{5} = \mathbf{0.2}$

$U_3 = \frac{5}{30} = \mathbf{0.17}$

$U_4 = \frac{2}{15} = \mathbf{0.13}$

$U_1 + U_2 + U_3 + U_4 = \mathbf{0.7}$

$0.7 < 0.7568$

**Will never miss a deadline
if RMA is applied!**

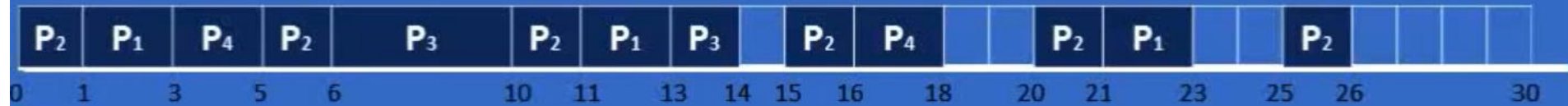
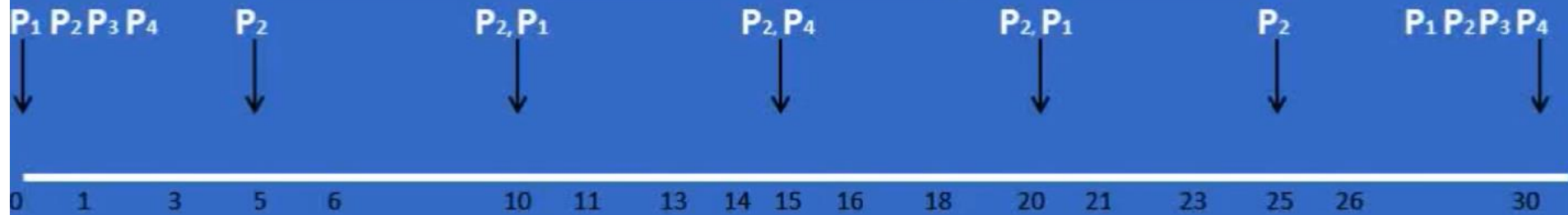
Process	Burst Time	Arrival Time	Period
P1	2	0	10
P2	1	0	5
P3	5	0	30
P4	2	0	15

Find the Least Common Multiple (LCM)

$$\begin{array}{c|c} 30 & 2 \\ \hline 15 & 3 \\ 5 & 5 \\ 1 & \\ \hline 2 \cdot 3 \cdot 5 & \end{array}
 \quad
 \begin{array}{c|c} 15 & 3 \\ \hline 5 & 5 \\ 1 & \\ \hline 3 \cdot 5 & \end{array}
 \quad
 \begin{array}{c|c} 10 & 2 \\ \hline 5 & 5 \\ 1 & \\ \hline 2 \cdot 5 & \end{array}
 \quad
 \begin{array}{c|c} 5 & 5 \\ \hline 1 & \\ \hline 5 & \end{array}$$

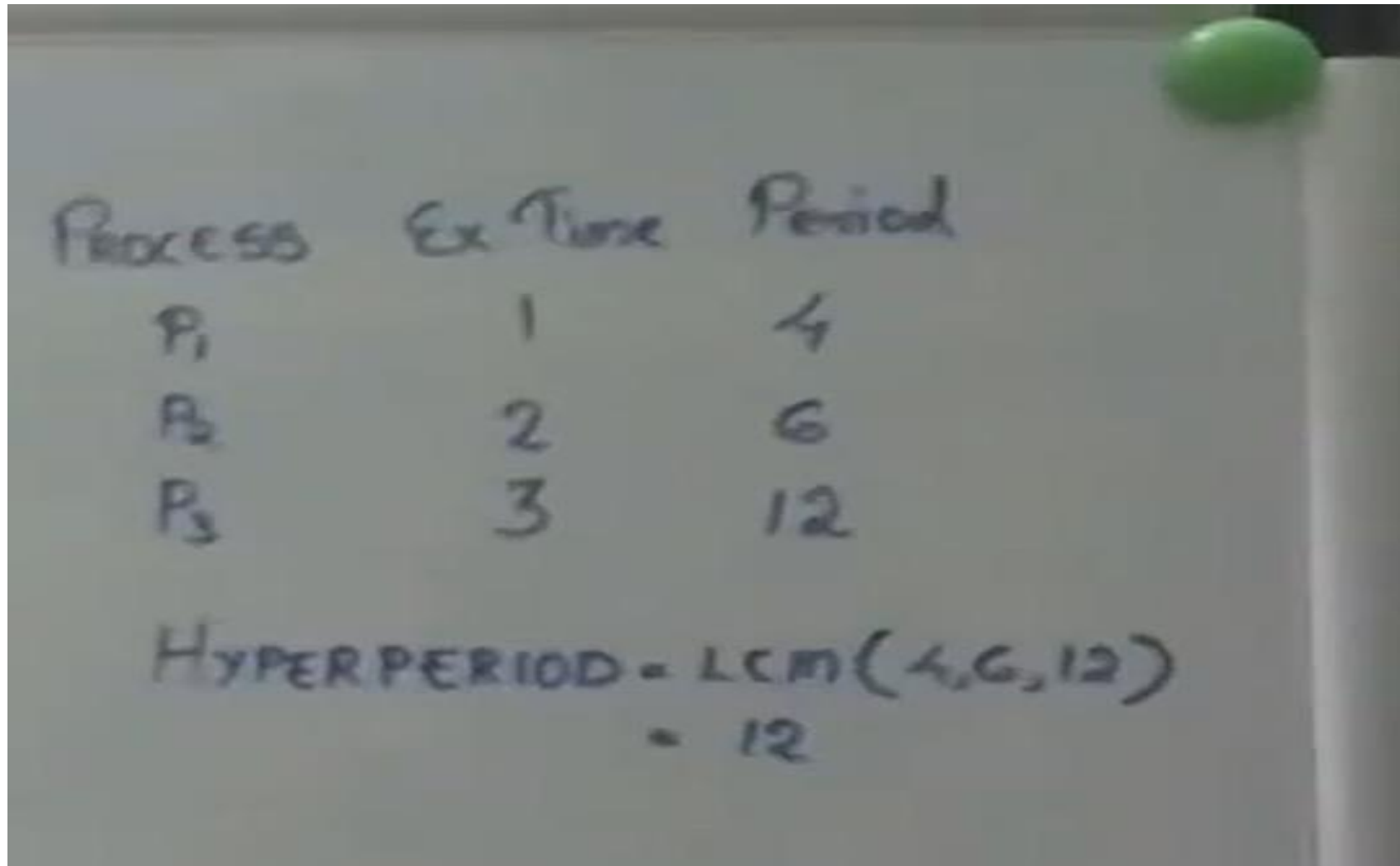
$\text{LCM} = 5 \cdot 3 \cdot 2$
 $= 30$
 $\text{GCD} = 5$

Process	Burst Time	Arrival Time	Period
P1	2	0	10
P2	1	0	5
P3	5	0	30
P4	2	0	15



Then it repeats

Example 3



Process	Ex Time	Period
P_1	1	4
P_2	2	6
P_3	3	12

$\text{HYPERPERIOD} = \text{LCM}(4, 6, 12)$
 $= 12$

Example 4

Process	Execution time	Period
P1	1	8
P2	2	5
P3	2	10

Least Upper Bound [\[edit\]](#)

The utilization will be: $\frac{1}{8} + \frac{2}{5} + \frac{2}{10} = 0.725$.

The sufficient condition for 3 processes, under which we can conclude that the system is schedulable is:

$$U_{lub} = 3(2^{\frac{1}{3}} - 1) = 0.77976 > 0.693 \dots \text{ (0.693 is utilization bound for RMS for } n = \infty \dots \text{)}$$

Because $0.77976 \geq 0.725$, and because being below the Least Upper Bound is a sufficient condition, the system is guaranteed to be schedulable.

Example 5

Process	Execution time	Period
P1	3	16
P2	2	5
P3	2	10

Least Upper Bound [\[edit\]](#)

Using the Liu and Layland bound, as in Example 1, the sufficient condition for 3 processes, under which we can conclude that the task set is schedulable, remains:

$$U_{lub} = 3(2^{\frac{1}{3}} - 1) = 0.77976$$

The total utilization will be: $\frac{3}{16} + \frac{2}{5} + \frac{2}{10} = 0.7875$.

Since $0.77976 < 0.7875$ the system is determined to *not* be guaranteed to be schedulable by the Liu and Layland bound.

Example 6

Process	Execution time	Period
P1	7	32
P2	2	5
P3	2	10

Least Upper Bound [\[edit \]](#)

Using the Liu and Layland bound, as in Example 1, the sufficient condition for 3 processes, under which we can conclude that the task set is schedulable, remains:

$$U_{lub} = 3(2^{\frac{1}{3}} - 1) = 0.77976$$

The total utilization will be: $\frac{7}{32} + \frac{2}{5} + \frac{2}{10} = 0.81875$.

Since $0.77976 < 0.81875$ the system is determined to *not* be guaranteed to be schedulable by the Liu and Layland bound.

Example 7

asks	Release time(r_i)	Execution time(C_i)	Deadline (D_i)	Time period(T_i)
T1	0	0.5	3	3
T2	0	1	4	4
T3	0	2	6	6

Example 7

- $U = 0.5/3 + 1/4 + 2/6 = 0.167 + 0.25 + 0.333 = 0.75$
- As processor utilization is less than 1 or 100% so task set is schedulable and it also satisfies the above equation of rate monotonic scheduling algorithm.

asks	Release time(r_i)	Execution time(C_i)	Deadline (D_i)	Time period(T_i)
T1	0	0.5	3	3
T2	0	1	4	4
T3	0	2	6	6

Example 8

Task	Execution Time	Period	Priority
T1	1	4	High
T2	2	6	Medium
T3	3	12	Low

Example 9

Task	Execution Time	Period	Priority
T1	1	3	High
T2	2.1	4	Low

Table 2: A set of tasks which are not RM schedulable

Example 10

Task	Execution Time	Period	Priority
T1	1	2	High
T2	2	5	Low

Table 3: A set of two tasks which doesn't meet the RMS sufficiency criteria, but is still RM schedulable

Rate Monotonic Scheduling g Algorithm

Process	Capacity (C)	Period (P)
T1	3	20
T2	2	5
T3	2	10

Rate Monotonic Scheduling Algorithm

Process	Capacity (C)	Period (P)
T1	3	20
T2	2	5
T3	2	10

- 1) How much time to schedule the algorithm
→ Scheduling Time = L.C.M. of Periods = 20
- 2) Priority
→ Least Period = High priority = $T2 > T3 > T1$

Rate Monotonic Scheduling Algorithm

Process	Capacity (C)	Period (P)
T1	3	20
T2	2	5
T3	2	10

a) For T2:

for every 5 period T2 has to execute 2 units

b) For T3:

for every 10 period T3 has to execute 2 units

c) For T1:

for every 20 period T1 has to execute 3 units

Process	Capacity (C)	Period (P)
T1	3	20
T2	2	5
T3	2	10

PRIORITY	PROCESS
1	T2
2	T3
3	T1

T1

T1 WILL EXECUTE 3 TIMES BETWEEN 1 TO 20 PERIODS

T3

T3 WILL EXECUTE 2 TIMES BETWEEN 1-10

T3 WILL EXECUTE 2 TIMES BETWEEN 10-20

T2

T2 WILL EXECUTE 2 TIME

T2 WILL EXECUTE 2 TIME

T2 WILL EXECUTE 2 TIME

T2 WILL EXECUTE 2 TIME

1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20
---	---	---	---	---	---	---	---	---	----	----	----	----	----	----	----	----	----	----	----

Process	Capacity (C)	Period (P)	PRIORITY	PROCESS
T1	3	20	1	T2
T2	2	5	2	T3
T3	2	10	3	T1

T1 WILL EXECUTE 3 TIMES BETWEEN 1 TO 20 PERIODS

T3 WILL EXECUTE 2 TIMES BETWEEN 1-10

T3 WILL EXECUTE 2 TIMES BETWEEN 10-20

T2 WILL EXECUTE 2 TIME

T2 WILL EXECUTE 2 TIME

T2 WILL EXECUTE 2 TIME

T2 WILL EXECUTE 2 TIME

T2	T2	T3	T3	T1	T2	T2	T1	T1		T2	T2	T3	T3		T2	T2			
1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20

Rate Monotonic Scheduling (RMS)

- **Schedulability check (off-line)**
 - A set of n tasks is schedulable on a uniprocessor by the RMS algorithm if the processor utilization (utilization test)

$$\sum_{i=1}^n c_i/p_i \leq n(2^{1/n} - 1).$$

c_i is the execution time and p_i is the period,
The term $n(2^{1/n} - 1)$ approaches $\ln 2$, (≈ 0.69 as $n \rightarrow \infty$).

- This condition is sufficient, but not necessary.

RMS Scheduler - Example 1

Task set: $T_i = (c_i, p_i)$ [computation time, period]

$T1 = (2, 4)$ and $T2 = (1, 8)$

Schedulability check:

$$2/4 + 1/8 = 0.5 + 0.125 = 0.625 \leq 2(\sqrt{2} - 1) = 0.82$$

RMS

$$\text{Rate} = \frac{1}{\text{period}} \rightarrow \text{Rate} \uparrow, \text{Period} \downarrow, \text{Priority} \uparrow$$

Q → $T_1(4, 1), T_2(5, 2)$ → Rate ↓, Period ↑, Priority ↓
 $T_3(20, 5)$

Task (P, e)
 $T_1 > T_2 > T_3$

~~2~~ ~~5~~ 4 2 0

